

Gestión y Optimización de Procesos en Linux mediante Docker y WSL2

Juan Stevan Cruz Díaz - 202459437

Jhoan Fabricio Hurtado - 202459472

Universidad del Valle – Sede Tuluá

Sistemas Operativos

Docente: Julián Enrique Castro

Resumen

La administración eficiente de procesos constituye uno de los pilares fundamentales de los sistemas operativos modernos. En este laboratorio se implementó un entorno controlado basado en contenedores Docker sobre Ubuntu ejecutado mediante WSL2 en Windows. Se utilizaron herramientas de monitoreo y generación de carga como htop, stress, stress-ng y cpulimit con el fin de analizar el comportamiento del sistema frente a distintos escenarios de consumo de CPU y memoria. Además, se realizaron actividades de gestión de procesos mediante señales de terminación, modificación de prioridades y limitación de recursos, permitiendo comprender mecanismos fundamentales empleados por Linux para la administración de procesos.

Palabras clave: Linux, Docker, procesos, CPU, memoria, htop, stress, cpulimit.

I. Introducción

Los sistemas operativos modernos deben administrar eficientemente los recursos de hardware disponibles para garantizar estabilidad y rendimiento. Entre sus funciones principales se encuentra la gestión de procesos, la cual permite controlar la ejecución concurrente de múltiples aplicaciones.

Linux ofrece diversas herramientas para monitorear, analizar y controlar procesos en ejecución. En este laboratorio se empleó un entorno Dockerizado que permitió realizar experimentos de carga controlada sin afectar el sistema anfitrión.

El objetivo principal consistió en comprender el comportamiento de los procesos bajo diferentes condiciones de ejecución y analizar cómo el sistema operativo distribuye los recursos disponibles.

II. Objetivos

Analizar el comportamiento de procesos, generar carga controlada, gestionar prioridades y aplicar limitaciones de recursos.

III. Metodología

El laboratorio fue desarrollado utilizando Ubuntu ejecutado sobre WSL2 y Docker Compose como plataforma de contenedorización.

Se creó un contenedor denominado monitor que incluyó las siguientes herramientas:

- htop
- stress
- stress-ng
- cpulimit
- procps
- python3

Adicionalmente se desarrolló un script en Python encargado de generar consumo artificial de CPU y memoria para observar el comportamiento del planificador del sistema operativo.

IV. Reconocimiento del Entorno

Inicialmente se realizó una inspección del sistema utilizando herramientas de monitoreo y consulta de procesos.

Se empleó el comando: htop

para visualizar en tiempo real el consumo de recursos del sistema.

Vista general del sistema utilizando htop.

Finalmente se utilizó:

pstree

para observar la jerarquía de procesos del sistema.

Árbol de procesos del sistema.

```
cruzd@JUANCruz:~/docker-lab$ pstree
systemd--2*[agetty]
      |
      |--cron
      |--dbus-daemon
      |--init-systemd(Ub)
      |   |--SessionLeader
      |   |   |--Relay(1004)
      |   |   |   |--docker-desktop--+
      |   |   |   |
      |   |   |   |--Relay(787)
      |   |   |   |   |--sh
      |   |   |   |   |--SessionLeader
      |   |   |   |   |   |--Relay(2203)
      |   |   |   |   |   |   |--bash
      |   |   |   |   |   |   |   |--pstree
      |   |   |   |   |   |   |
      |   |   |   |   |   |   |   |--init
      |   |   |   |   |   |   |   |   |--{init}
      |   |   |   |   |   |   |   |   |--2*[login]
      |   |   |   |   |   |   |   |   |   |--bash
      |   |   |   |   |   |   |   |   |   |--{init-systemd(Ub)}
      |   |   |   |   |   |   |   |   |
      |   |   |   |   |   |   |   |   |   |--polkitd
      |   |   |   |   |   |   |   |   |   |   |--3*[{polkitd}]
      |   |   |   |   |   |   |   |   |   |   |--rsyslogd
      |   |   |   |   |   |   |   |   |   |   |   |--3*[{rsyslogd}]
      |   |   |   |   |   |   |   |   |   |   |--snapd
      |   |   |   |   |   |   |   |   |   |   |   |--13*[{snapd}]
      |   |   |   |   |   |   |   |   |   |   |   |--3*[snapfuse]
      |   |   |   |   |   |   |   |   |   |   |   |   |--2*[{snapfuse}]
      |   |   |   |   |   |   |   |   |   |   |   |   |--snapfuse
      |   |   |   |   |   |   |   |   |   |   |   |   |   |--6*[{snapfuse}]
      |   |   |   |   |   |   |   |   |   |   |   |   |   |--2*[systemd]
      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |--(sd-pam)
      |   |   |   |   |   |   |   |   |   |   |   |   |   |--systemd-journal
      |   |   |   |   |   |   |   |   |   |   |   |   |   |--systemd-logind
      |   |   |   |   |   |   |   |   |   |   |   |   |   |--systemd-resolve
      |   |   |   |   |   |   |   |   |   |   |   |   |   |--systemd-timesyn
      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |--{systemd-timesyn}
      |   |   |   |   |   |   |   |   |   |   |   |   |   |--systemd-udev
      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |--5*[{udev-worker}]
      |   |   |   |   |   |   |   |   |   |   |   |   |   |--unattended-upgr
      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |--{unattended-upgr}
      |   |   |   |   |   |   |   |   |   |   |   |   |
      |   |   |   |   |   |   |   |   |   |   |   |   |   |--cruzd@JUANCruz:~/docker-lab$ |
```

V. Saturación de CPU

Con el objetivo de analizar el comportamiento del procesador bajo carga, se utilizó la herramienta stress.

Se ejecutó: stress --cpu 4 --timeout 60s

La instrucción generó cuatro procesos consumidores de CPU durante un minuto.

Durante la ejecución se observó un incremento significativo en el porcentaje de utilización de los núcleos disponibles.

```
cruzd@JUANCruz:~/docker-lab$ stress --cpu 4 --timeout 60s
stress: info: [2420] dispatching hogs: 4 cpu, 0 io, 0 vm, 0 hdd
stress: info: [2420] successful run completed in 60s
```

Los resultados evidenciaron que Linux distribuye los procesos de carga entre los núcleos disponibles para maximizar el aprovechamiento del hardware.

VI. Saturación de Memoria

Se generó consumo artificial de memoria utilizando: `stress --vm 2 --vm-bytes 256M --timeout 60s`

El sistema reservó memoria progresivamente hasta alcanzar el límite establecido.

```
cruzd@JUANCRUZ:~/docker-lab$ stress --vm 2 --vm-bytes 256M --timeout 60s
stress: info: [2443] dispatching hogs: 0 cpu, 0 io, 2 vm, 0 hdd
stress: info: [2443] successful run completed in 61s
```

Se verificó el estado de la memoria mediante:

`free -h`

```
cruzd@JUANCRUZ:~/docker-lab$ free -h
              total        used        free      shared  buff/cache   avail
Mem:           6.7Gi       1.3Gi       3.0Gi         16Mi       2.5Gi       5
Swap:          2.0Gi          0B       2.0Gi
```

Los resultados mostraron una disminución de la memoria libre y un incremento temporal del uso de RAM.

VII. Competencia entre Procesos

Para observar el comportamiento del planificador se ejecutaron simultáneamente múltiples procesos consumidores de recursos:

`stress --cpu 2 &`

`python3 -c "while True: pass" &`

```
GNU nano 7.2 script_cpu_ram.py *
import threading

data = []

def consumir_cpu():
    while True:
        pass

def consumir_ram():
    while True:
        data.append("X" * 1000000)

for i in range(4):
    threading.Thread(target=consumir_cpu).start()

consumir_ram()
```

dd if=/dev/zero of=/dev/null bs=1M &

Los procesos compitieron por el acceso al procesador y a los recursos del sistema.

Main		I/O											
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command		
2429	cruzd	20	0	5648	4096	3200	R	0.7	0.1	0:00.07	htop		
1	root	20	0	21940	12392	9192	S	0.0	0.2	0:01.82	init		
2	root	20	0	3120	1792	1792	S	0.0	0.0	0:00.02	init		
6	root	20	0	3136	1796	1792	S	0.0	0.0	0:00.00	plan9 --con		
7	root	20	0	3136	1796	1792	S	0.0	0.0	0:00.00	plan9 --con		
8	root	20	0	3120	1792	1792	S	0.0	0.0	0:00.01	init		
45	root	19	-1	66820	19056	18032	S	0.0	0.3	0:00.72	systemd-jou		
92	root	20	0	25412	6272	4736	S	0.0	0.1	0:01.37	systemd-ude		
104	root	20	0	149M	1412	1280	S	0.0	0.0	0:00.00	snapfuse /v		
105	root	20	0	149M	1412	1280	S	0.0	0.0	0:00.00	snapfuse /v		
106	root	20	0	149M	1412	1280	S	0.0	0.0	0:00.00	snapfuse /v		
107	root	20	0	441M	12624	1408	S	0.0	0.2	0:00.00	snapfuse /v		
108	root	20	0	441M	12624	1408	S	0.0	0.2	0:00.60	snapfuse /v		
109	root	20	0	441M	12624	1408	S	0.0	0.2	0:00.61	snapfuse /v		
111	root	20	0	149M	1540	1408	S	0.0	0.0	0:00.00	snapfuse /v		
112	root	20	0	149M	1540	1408	S	0.0	0.0	0:00.00	snapfuse /v		
113	root	20	0	149M	1540	1408	S	0.0	0.0	0:00.00	snapfuse /v		
115	root	20	0	149M	1668	1408	S	0.0	0.0	0:00.00	snapfuse /v		
116	root	20	0	149M	1668	1408	S	0.0	0.0	0:00.00	snapfuse /v		
117	root	20	0	149M	1668	1408	S	0.0	0.0	0:00.00	snapfuse /v		
179	systemd-re	20	0	21460	12800	10624	S	0.0	0.2	0:00.25	systemd-res		
181	systemd-ti	20	0	91028	7680	6784	S	0.0	0.1	0:00.24	systemd-tim		
194	systemd-ti	20	0	91028	7680	6784	S	0.0	0.1	0:00.00	systemd-tim		
196	root	20	0	4236	2432	2304	S	0.0	0.0	0:00.02	cron -f -P		
197	messagebus	20	0	9640	4608	4224	S	0.0	0.1	0:00.24	@dbus-daemo		
209	root	20	0	2168M	38584	25216	S	0.0	0.6	0:00.38	snappd		
214	root	20	0	18156	8064	7296	S	0.0	0.1	0:00.19	systemd-log		
223	root	20	0	3160	1920	1792	S	0.0	0.0	0:00.00	agetty -o -		
225	syslog	20	0	217M	5248	4224	S	0.0	0.1	0:00.05	rsyslogd -n		
233	root	20	0	3116	1792	1664	S	0.0	0.0	0:00.01	agetty -o -		
245	root	20	0	104M	22136	13056	S	0.0	0.3	0:00.28	python3 /us		
F1Help F2Setup F3SearchF4FilterF5Tree F6SortByF7Nice -F8Nice +F9Kill F10Q													

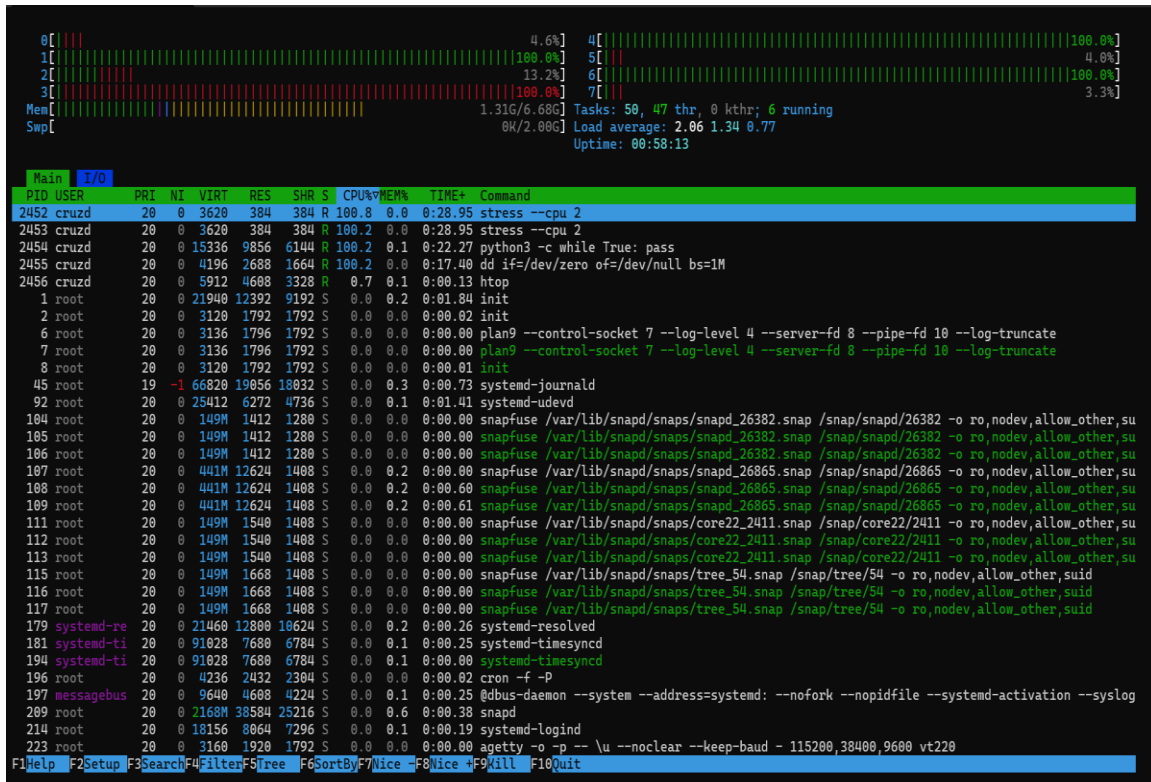
Durante la prueba se evidenció cómo Linux distribuye equitativamente el tiempo de CPU entre los procesos activos.

VIII. Gestión de Procesos

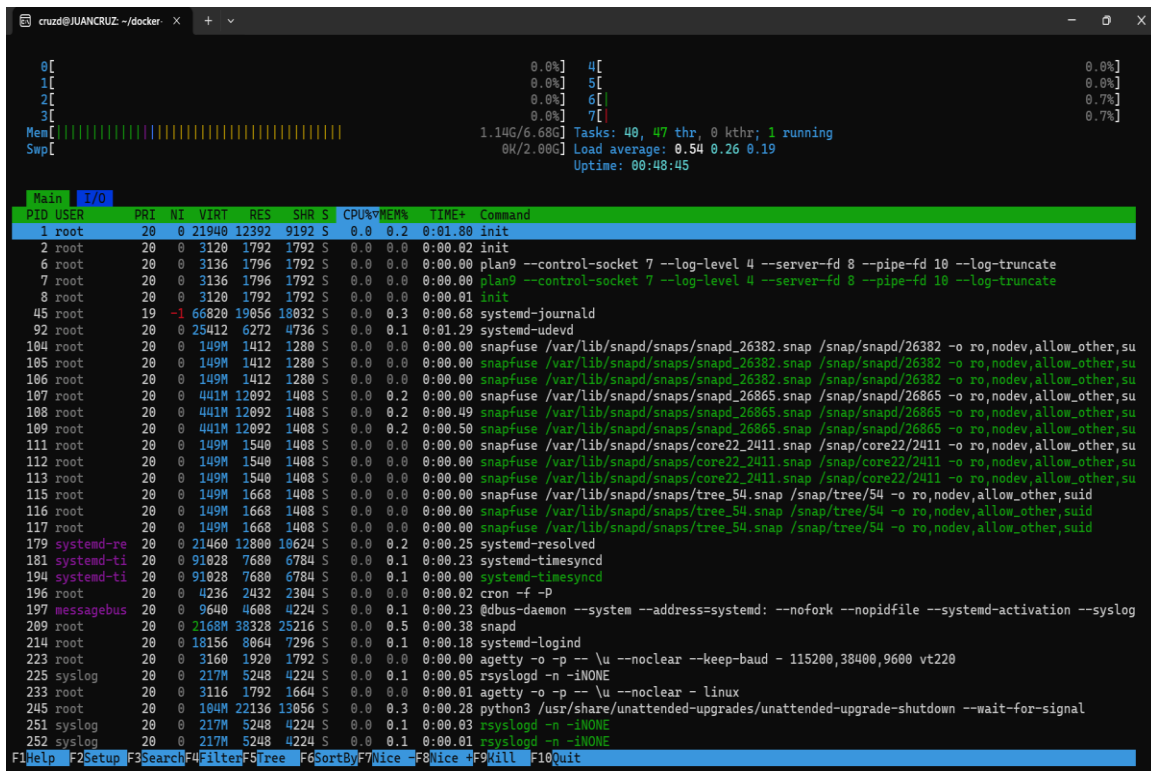
Se identificaron procesos mediante: ps aux

Posteriormente se utilizaron señales de terminación: kill PID y kill -9 PID

Procesos antes de aplicar kill.



Procesos después de aplicar kill.



Análisis

El comando kill envía una señal SIGTERM permitiendo que el proceso finalice de manera controlada. Por otro lado, kill -9 envía SIGKILL, forzando la terminación inmediata del proceso sin permitir tareas de limpieza.

IX. Gestión de Prioridades

Se realizaron pruebas modificando prioridades mediante nice.

Proceso con baja prioridad: nice -n 19 stress --cpu 2 &

```
cruzd@JUANCRUZ:~/docker-lab$ nice -n 19 stress --cpu 2 &
[6] 2466
cruzd@JUANCRUZ:~/docker-lab$ stress: info: [2466] dispatching hogs: 2 cpu, 0
io, 0 vm, 0 hdd
```

Proceso con prioridad elevada: sudo nice -n -5 stress --cpu 2 &

```
cruzd@JUANCRUZ:~/docker-lab$ sudo nice -n -5 stress --cpu 2 &
[8] 2482
cruzd@JUANCRUZ:~/docker-lab$ stress: info: [2484] dispatching hogs: 2 cpu, 0
io, 0 vm, 0 hdd
```

Análisis

Los procesos con valores nice menores recibieron mayor tiempo de CPU. En contraste, los procesos con valores positivos altos cedieron recursos a otros procesos del sistema.

X. Limitación de Recursos

Se utilizó cputime para restringir el consumo de CPU de un proceso específico: cputime -p PID -l 30

Los resultados demostraron que la herramienta permite controlar el uso de recursos sin finalizar el proceso.

XI. Preguntas de Análisis

1. ¿Cuál es la diferencia entre kill y kill -9? ¿Cuándo usarías cada uno?

- kill utiliza SIGTERM y permite finalizar procesos de forma controlada; kill -9 utiliza SIGKILL y fuerza la terminación inmediata.

2. ¿Qué pasó con el proceso de mayor prioridad vs el de menor prioridad durante la carga?
 - El proceso con mayor prioridad recibió más tiempo de CPU durante la carga.
3. ¿En qué situaciones sería preferible bajar la prioridad de un proceso en lugar de matarlo?
 - Es preferible disminuir la prioridad cuando el proceso sigue siendo necesario para el sistema.
4. ¿Qué indica la columna NI en **htop**? ¿Cuál es su rango?
 - NI representa el valor nice. Su rango va desde -20 hasta 19.
5. ¿Qué es un proceso zombi y por qué es un problema?
 - Un proceso zombi es un proceso finalizado cuya información permanece en la tabla de procesos hasta que el padre recoge su estado.

XII. Conclusiones

El laboratorio permitió comprender los mecanismos fundamentales utilizados por Linux para la administración de procesos.

Las pruebas realizadas demostraron la importancia de herramientas de monitoreo como **htop** para observar el comportamiento del sistema en tiempo real.

Asimismo, se evidenció que Linux proporciona mecanismos robustos para controlar procesos mediante señales, prioridades y limitación de recursos, garantizando estabilidad y eficiencia en ambientes multitarea.

Finalmente, la utilización de Docker permitió realizar las pruebas de forma segura, aislada y reproducible

XIII. Repositorio

Repositorio GitHub del proyecto:

<https://github.com/JuanCruzUV12/docker-lab>